手把手教你养龙虾|openclaw 纯干货操作指南（一）

声明：OpenClaw 基于 Windows 系统（WSL2 架构）的本地部署与配置规范

赘述：

随着人工智能从云端大模型向本地化自主智能体（Autonomous Agents）演进，OpenClaw 作为一种基于网关架构构建的开源 AI 平台，凭借其跨平台消息整合能力引发了广泛的技术关注。鉴于 Windows 原生环境在处理 POSIX 标准系统调用时存在底层架构差异，本研究与部署指南确立了以第二代 Windows 平台 Linux 子系统（WSL2）为核心运行环境的技术路线。该架构方案旨在有效保障底层网络通信的稳定性与异步 I/O 的高效响应，为在 Windows 宿主机上搭建高可用的私人 AI 助手提供标准化的操作路径。

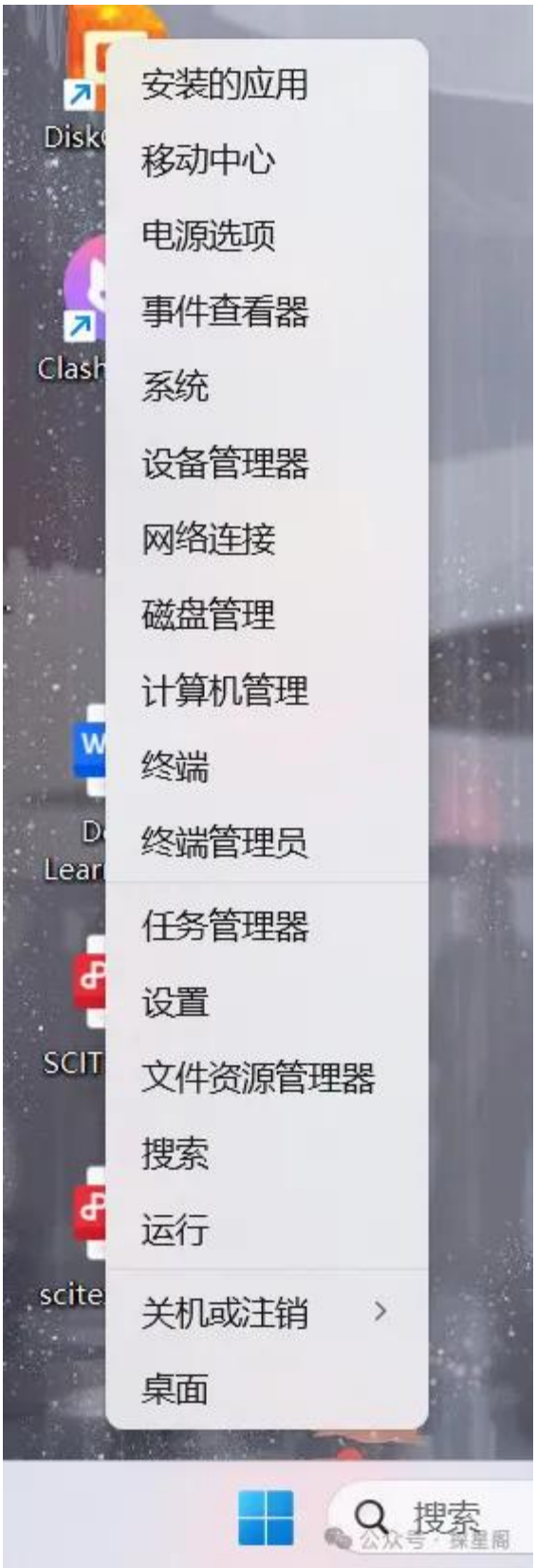
本节目录：

1. **虚拟化子系统的挂载与底层运行环境构建**
2. **核心网关实例化与跨网络层穿透机制**
3. **零信任架构下的安全握手与日常唤醒规范**
4. **运行时高频异常与深度干预指南**
5. **大语言模型架构的拓展与异构系统挂载**
6. **零信任架构下的本地网络隔离与安全加固**
7. **极端态势下的环境净空与底层重构**

1. 虚拟化子系统的挂载与底层运行环境构建

构建可靠运行环境的基石在于虚拟化子系统的正确挂载。研究人员需依托 Windows 系统内置的 Hyper-V 架构特征，激活并部署 WSL2 及配套的 Linux 发行版（本指南以公认稳定性较高的 Ubuntu 为例）。系统级的干预需在具备管理员权限的 PowerShell 终端中展开。

操作：右键任务栏的 Windows 窗口，点击终端管理员，打开具备管理员权限的 PowerShell 终端（如图 1）。



终端键入（逐个复制粘贴即可）：

```
# 触发 WSL 核心组件及默认 Ubuntu 发行版的自动化安装流
wsl --install
# 强制将 WSL 的默认架构版本锁定为第二代，以获取完整的系统调用能力
wsl --set-default-version 2
```

然后重启电脑，再次按上述操作打开终端管理员将自动唤起 Ubuntu 终端界面。

如未自动唤起 Ubuntu 终端界面，

可能的原因：WSL（Windows 子系统）的核心组件已经启用，但**尚未成功安装具体的 Linux 发行版（如 Ubuntu）**。这通常是因为之前执行 wsl --install 时，遇到网络波动或系统自带应用商店的连接问题导致下载中断。

操作如下：

直接在当前的 PowerShell 窗口中，执行以下命令来强制指定安装 Ubuntu 系统：**wsl --install -d Ubuntu**

敲击回车后，系统会开始下载并安装 Ubuntu，这可能需要几分钟的时间（取决于网速）。
安装完成后，它会自动弹出一个新的 Ubuntu 终端窗口（或者提示您已安装完成，您可以再次输入 wsl 进入）。
首次进入时，系统会提示您创建一个 Linux 本地账户。
Enter new UNIX username: 输入您想要的用户名（建议使用纯小写英文字母，如 admin 或您的名字缩写）。
New password: 输入密码（注意：**输入密码时屏幕上不会显示任何字符**，这是 Linux 的安全机制，盲输完直接按回车即可）。
Retype new password: 再次输入密码确认。
完成上述账户初始化后，您就可以看到绿色的命令提示符（比如 username@hostname:~$），这代表 Ubuntu 已经就绪。此时，您可以继续进行 Node.js 的安装了。

Node.js 的安装：

伴随底层虚拟化子系统的成功构建并设定独立的安全凭证后，后续环境依赖的配置工作将全面迁移至 Ubuntu 终端内展开。由于 OpenClaw 的并发任务处理机制对 Node.js 的运行时版本存在严格限制（强制要求 v22 或更高版本），直接使用系统默认包管理器往往难以获取最新版本。为实现对运行环境的精准控制，研究人员应引入 Node 版本管理器（NVM）实施依赖注入。

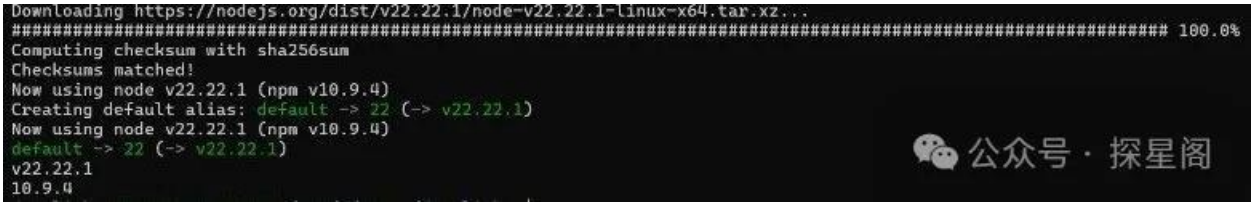
针对这一版本控制需求，引入 Node 版本管理器（NVM）是当前学术界与工程实践中的标准规范。请在您当前所在的 Ubuntu 终端内，逐行复制并执行以下指令流，以完成运行时环境的精准注入：

```
# 获取并执行NVM安装脚本，在当前用户目录下建立版本控制体系
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
# 重新加载环境变量配置文件，确保系统能够即时识别NVM指令source ~/.bashrc
# 引入并激活符合OpenClaw版本强制要求的Node.js环境
nvm install 22nvm use 22nvm alias default 22
# 验证当前底层依赖的安装状态与具体版本号
node -vnpm -v
```

在以往的本地化模型部署测试中，我们观察到通过 curl 拉取 GitHub 托管的外部脚本时，偶发的区域性网络阻滞可能导致下载进程挂起或中断。

执行上述指令期间，请**确保宿主机网络状态稳定。**

待最后两条验证指令（node -v 与 npm -v）在终端中成功返回以 **v22.开头的版本标识**后，即可宣告本阶段环境构建工作的圆满达成。届时，我们将依照部署指南继续推进核心框架的拉取与网关参数的实例化工作。



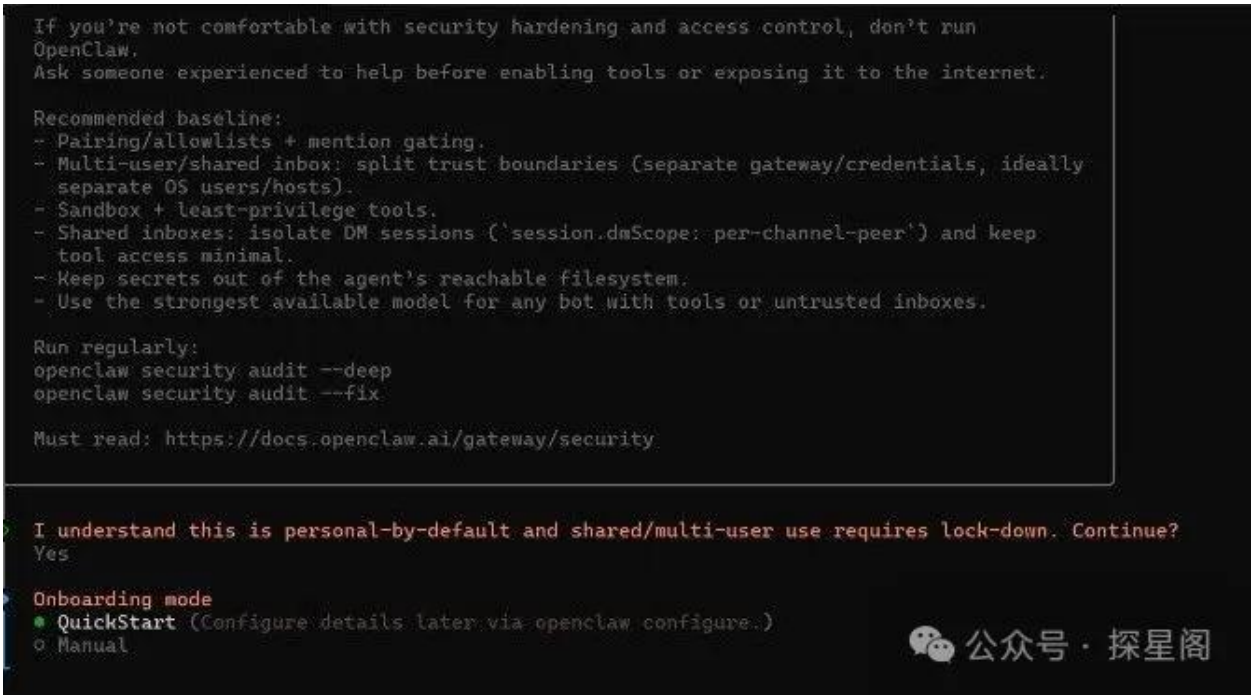
2. 核心网关实例化与跨网络层穿透机制

基础运行时确立之后，部署工作的重心随即转向 OpenClaw 框架的拉取与参数实例化。

执行自动化构建脚本，请在您当前的 Ubuntu 终端内（即带有绿色提示符的运行视界），完整复制并执行以下网络请求指令：

```
curl -fsSL https://openclaw.bot/install.sh | bash
```

系统将映射出一个交互式的终端用户界面（TUI）。在此阶段，需确认部署范式为“QuickStart”，并在大语言模型（LLM）挂载环节完成 API 密钥的初始授权。



界面是纯文本终端界面，不支持鼠标点击，请您完全使用**键盘**来进行后续操作，具体步骤如下：

确认快速启动：目前绿色的焦点已经停留在 QuickStart 上，您只需要直接按下键盘上的 **Enter (回车键)**。

选择大语言模型 (LLM)：按下回车后，界面会跳转到模型选择列表。请使用键盘的**上下方向键**，选择一个您拥有 API Key 的模型服务商（例如推荐使用国内访问稳定的 Qwen 通义千问，或者 Anthropic/OpenAI）。选好后按回车。

输入 API 密钥：系统会提示您输入所选模型的 API Key。请复制您的 Key，然后在 Ubuntu 终端窗口中**单击鼠标右键**（这是终端默认的粘贴快捷键，或者使用 Ctrl+Shift+V）将其粘贴进去，按下回车。*(注意：输入或粘贴密码/Key 时，屏幕上可能不会显示字符，这是正常的安全机制)*。

配置 Channels（通讯渠道）：当系统询问是否配置第三方通讯平台（如 Telegram 等）时，建议先选择 **“Skip for now（暂时跳过）”** 并按回车，先确保本地网关能跑通。

配置 Hooks（生命周期钩子）：如果出现带括号 [ ] 的多选列表，请使用键盘的 **空格键 (Space)** 来打勾。建议找到并勾选 command-logger 和 session-memory，全部勾选完毕后，按回车键确认。

（两个开局模式的区别，不看略过：

在 OpenClaw 框架的实例化阶段，系统提供的 QuickStart（快速启动）与 Manual（手动配置）两种初始化模式，其核心差异体现在配置维度的颗粒度以及系统接管预设参数的层级上。

QuickStart 模式主要面向快速原型构建与基础环境的可用性验证。采用该模式时，框架会自动注入一套经过官方充分校验的基准预设参数，从而在前端视图中屏蔽底层繁杂的网络映射与存储协议配置。在此路径下，系统仅要求操作者交互式地补充最为核心的运行凭证（尤其是大语言模型的类型选取及其 API 密钥绑定）。这种预设前置的设计策略大幅缩减了初始部署的时间成本，使研究人员能够基于最少

的信息输入，迅速建立一个最小可行性网关（Minimum Viable Gateway）。待基础的本地模型推理链路打通后，架构内的其他非核心参数均可通过后续调用的 openclaw configure 指令进行按需延展与微调。

与上述敏捷部署逻辑相对，Manual 模式则专为需要实现底层架构深度定制的生产环境或高阶实验场景而设计。触发此模式后，安装向导将彻底放弃默认参数的隐式注入，转而逐一遍历网关服务的所有可控变量。操作者必须在系统初始化阶段，显式声明各项底层通信与存储规则，其涵盖范围通常包括自定义数据库的持久化路径、跨域资源共享（CORS）的安全策略限制、网关监听端口的异构映射，以及各类第三方通信渠道（Channels）的回调钩子（Webhook）绑定。尽管该模式显著增加了前期环境部署的技术开销，但其确保了网关架构自启动之初，便严密贴合宿主机特定的网络拓扑限制与数据合规要求。

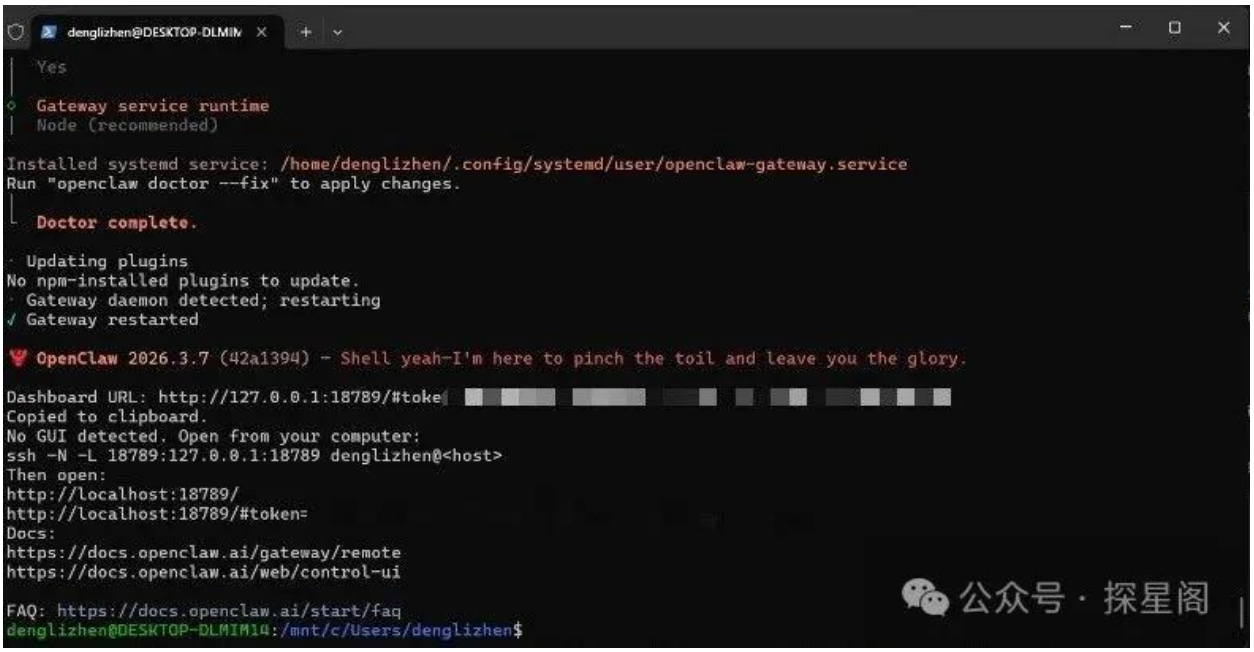
结合当前的本地化基准测试环境，对于首次尝试搭建并验证私人 AI 助手联通性的场景，采用 QuickStart 模式足以跑通底层的异步 I/O 通信与大模型 API 调用逻辑，这也是目前部署规范中所推荐的标准操作路径。您可以直接在终端按下回车键以确认该默认选项，并继续进行后续的模型授权绑定工作。）

初始化序列终结后，OpenClaw 会自动将其核心网关注册为 Linux 子系统级别的守护进程（systemd service）。然而，在调用境外大语言模型（如 Google Gemini 系列）的实际推理场景中，WSL2 架构固有的 NAT（网络地址转换）隔离机制会导致子系统无法直接继承 Windows 宿主机的代理上下文，进而引发网关握手阶段出现物理阻断。

注：建议先弄个国产 AI 试试，如 KIMI 就行，API 好搞而且不怎么花钱，不然很容易花钱又连不上 balabala 的

为确立稳定的跨网络层通信链路，实施**基于 TUN 接口的全局网络层接管**构成了最具鲁棒性的技术解法。研究人员需在 Windows 宿主机运行的代理辅助工具（如 Clash Verge）中，激活“虚拟网卡模式（TUN Mode）”。该模式成功接管内核后，宿主机的全局路由表将被重构，所有源自 WSL2 内部发出的外网访问流量均将被物理截获并无缝导入加密隧道，从而有效规避异步通信中的结构性丢包与超时异常。

最后样式



（我的 token 就不露出来了）

当前系统已自动将网关注册为 Linux 子系统级别的后台服务（systemd service），并动态生成了用于首次越权访问的控制平面入口。为了实现从终端命令行向图形化交互视界的跨越，我们需执行部署流的最终核验步骤。

针对您当前的运行视界，请执行以下具体操作以完成闭环：

请使用鼠标在您的 Ubuntu 终端窗口内，仔细框选并复制这一整行包含极长哈希值的地址链接：类似于 http://127.0.0.1:18789/#token=FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF

（请务必确保复制的范围包含了最后的字母 d3，不得有遗漏）。提取该链路后，请直接将其粘贴至您 Windows 系统常规使用的浏览器地址栏中并敲击回车。

这种依托专属 Token 实现首次握手的机制，有效替代了常规的命令行人工审核环节。一旦浏览器成功加载该页面，您将直接步入 OpenClaw 的图形化控制中枢，进而开启与 Gemini 大模型本地化联动的实质性研究工作。如果您在浏览器访问期间遭遇“连接被拒绝”等网络重定向异常，请随时向我反馈。

好了，你可以使用对话窗口调教龙虾了。

以下属于日常维护和相关问题：

3. 零信任架构下的安全握手与日常唤醒规范

在宿主机经历物理断电、系统重启或终端视界被人工关闭后，运行于 WSL2 架构内部的守护进程通常会随之挂起。为恢复对本地化大语言模型的调度能力，操作者需依循严谨的初始化逻辑，重新唤起子系统并重建前端握手通信。

第一阶段：进程健康度核验与分页器干预

重新唤起 Ubuntu 终端后（应用栏直接搜索 Ubuntu 启动即可），由于 WSL2 存在冷启动延迟，研究人员需立即核验核心守护进程的真实运转状态：

```
systemctl --user status openclaw-gateway.service
```

若终端回显信息中包含绿色的 active (running) 标识，即证明底层异步通信模块已处于就绪状态。此时，终端视界通常会被底层系统文本分页器（如 less）全面接管，并在屏幕底部呈现类似 Lines 1-20/20 (END) 的锁定标识。为释放命令行输入端口以执行后续操作，操作者**必须且仅能敲击键盘的 q 键**以强制中止状态审阅。

第二阶段：底层通信凭证的直接抽离

跨端访问的拦截机制源于零信任架构下动态哈希令牌（Token）的易失性。系统内核的周期性重启必然导致前期缓存在前端的鉴权凭证彻底失效。由于标准输出管道（Pipeline）存在不可控的视觉截断效应（如日志末尾出现 > 符号隐匿了关键字符串），操作者必须绕过日志系统，直接对本地持久化存储实施正则提取：

```
grep -i "token" ~/.openclaw/openclaw.json
```

系统将精准剥离出隐藏的一段纯粹哈希字符串。将其严丝合缝地拼补至网关基础地址的尾端（构成形如 http://127.0.0.1:18789/#token=提取出的长字符串 的格式），并覆写至 Windows 浏览器的导航栏并发起请求，方能从根本上消解应用层前端抛出的 unauthorized: gateway token missing 非授权拦截壁垒。

4. 运行时高频异常与深度干预指南

在网关运转与模型高频调用期间，受限于服务商的风控策略与本地环境的路径偏移，系统可能会触发一系列深层连锁故障。研究人员应依据以下现象拓扑图进行结构性修复。

4.1 接口频控风控（API Rate Limit Reached）与双轨架构降级

当系统向服务端提交的推理请求遭遇严苛的频控拦截时，必须实施大模型架构的降级操作（如从庞大的 3.1-pro-preview 迁移至敏捷型的 2.5-flash）。由于 OpenClaw 在设计上严格区隔了“全局网关基准”与“独立智能体（Agent）工作区”，参数覆写必须在 CLI 与 GUI 层面实施双轨并行。

终端全局映射覆写：在 Ubuntu 终端下达

```
openclaw configure
```

指令，利用键盘方向键与空格键选中 Model 模块，将默认引擎重置为 google/gemini-2.5-flash 并确认归档孤立数据碎片（Orphan transcripts）。

注：出现这个问题因为我贪心的使用了 3.1-pro-preview，导致无法正常使用。

应用层实例状态强制剥离：转入浏览器控制面板的“代理（Agent）”模块。定位至“主模型（默认）”输入域，**无视任何前端渲染的冗余下拉联想列表，直接利用物理键盘的退格键（Backspace）将既存的**

高消耗标识逐字符彻底抹除。在确保文本框完全净空后，手动精确键入 google/gemini-2.5-flash，并务必点击右下方的红色“保存”控件。
环境净空：切回“聊天”主面板，强制点击“新一届会议（New session）”以斩断附带旧参数的内存污染流。

4.2 诊断修复引发的拓扑变量丢失与进程死锁（ERR_CONNECTION_REFUSED）

在前端网址抛出拒绝连接异常时，若底层 journalctl 审计日志高频反馈 Gateway start blocked: set gateway.mode=local (current: unset) 及 status=1/FAILURE，即表明前期执行的环境强制修复（如

```
openclaw doctor --fix
```

）意外破坏了配置文件中的物理宿主声明变量。基于异常隔离机制，网关引擎陷入了无限循环的启动崩溃态。

破局路径：研究人员必须从系统命令行接口强行重构网关的拓扑环境变量。下达

```
openclaw configure
```

指令唤醒交互向导，当界面抛出“Where will the Gateway run?”的拓扑质询时，**务必将焦点锚定于** Local (this machine) **并敲击回车键予以硬性确认。**待后续冗余配置项快速跳过并输出 Configure complete 后，下达

```
systemctl --user restart openclaw-gateway.service
```

指令迫使内核加载全新配置，系统即可彻底挣脱死锁状态。

5. 大语言模型架构的拓展与异构系统挂载

为适应多样化的下游科研任务，研究人员必须掌握向本地网关注入多源异构模型（如 OpenAI GPT、Anthropic Claude 系列）的标准化流程。

服务商加密凭证注入：在底层 Ubuntu 终端内（确保已退出分页器锁定），依据服务商类别下达密钥装载指令。例如：

```
openclaw keys add openai <您的专属密钥>
```

内置安全组件将自动对凭证执行加密并写入本地鉴权库。

实例模型标识符映射：切回浏览器图形化界面的“代理”模块，在“主模型”输入域内运用退格键清空旧数据，并严格采用 服务商代号/具体模型代号 的命名范式实施填报（例如录入 openai/gpt-4o 或 anthropic/claude-3-5-sonnet-20241022）。点击保存并开启新会话后，智能体即刻依附于全新的算力底座。

6. 零信任架构下的本地网络隔离与安全加固（重要）

伴随高权限 API 密钥的本地持久化落盘，实施结构化的网络防护已成为保障数据资产安全的必要前提（3.8 也更新了相关内容）。

物理文件层的提权锁定： OpenClaw 的全局环境变量以高密级 JSON 格式驻留于底层。为阻断宿主环境内潜在的第三方跨用户态越权读取，操作者必须调用 POSIX 指令强制收敛访问权限：

```
chmod 600 ~/.openclaw/openclaw.jsonchmod 700 ~/.openclaw
```

此举将操作系统访问控制列表（ACL）严格重构，仅允许所有者执行读写操作。

WSL2 局域网暴露面的收敛矩阵： 基于 WSL2 的动态端口映射机制，网关的默认监听端口（ ）会被穿透至宿主机。为抵御源自同局域网（LAN）内其他物理终端的非法嗅探，操作者需在 Windows 的高级安全防御面板（wf.msc）中构建高优先级入站规则：**明确拒绝所有源 IP 非本机环回地址（**127.0.0.1**）（以本机为例），且尝试向本地 TCP 端口发起握手的网络请求。**

7. 极端态势下的环境净空与底层重构（俗称重开）

面对无法通过常规手段排解的深层配置冲突或库文件破损，执行全局卸载与重新部署构成了最为彻底的技术干预方案。这一重构序列必须严格遵循先剥离内存占用、后物理销毁数据的逻辑次序：

代理通信隧道的 DNS 防泄漏： 为确保底层网关向云端集群发起推理请求的绝对匿名性，操作者需在 Clash Verge 等代理客户端的系统设置内，强制激活 **Strict Route（严格路由）** 并将 DNS 解析模式配置为 **Fake-IP**。这一拦截机制有效防止了真实域名解析轨迹向本地宽带运营商（ISP）的溢出泄漏。

```
# 切断后台守护进程的内存占用与执行线程
systemctl --user stop openclaw-gateway.service
# 剥离系统层自启动注册项，防止内核意外拉起残留进程
systemctl --user disable openclaw-gateway.service
# 强制抹除包含身份凭证、实例参数及历史缓存的全局工作区目录（数据不可逆转）
rm -rf ~/.openclaw
```

待底层文件系统彻底回归纯净的初始基态后，研究人员方可复用自动化拉取脚本

```
curl -fsSL https://openclaw.bot/install.sh | bash
```

重启新一轮的标准化部署与参数调配进程。

这篇文章会同步更新到官网，欢迎查看。